PIC Deposition Benchmark Suite: A Reproducible Reference for Charge Deposition in Two-Dimensional Electrostatic Particle-In-Cell Simulations

Elhadj Hocine[a]

[a]*Independent Researcher*

Abstract

Charge deposition dominates cost and accuracy in explicit Particle-In-Cell (PIC) plasma simulations. We present an open-source 2D electrostatic reference program that evaluates Nearest Grid Point (NGP), Cloud-In-Cell (CIC), Triangular-Shaped Cloud (TSC), and Esirkepov deposition under cache-aware CPU optimizations and optional CUDA backends. The suite separates cell-sort cost from deposit-kernel throughput, profiles full PIC timesteps (push, sort, deposit, Poisson solve, gather), and bundles charge, energy, spectral-noise, Langmuir-wave, two-stream, and long-run conservation validation. Benchmarks on Apple M1 Pro and Tesla T4 (same-host CPU baseline) show deposition is memory-bandwidth limited; CIC offers the best CPU accuracy–performance trade-off and GPU atomics are preferred at large particle counts on T4. Archived CSV pipelines regenerate all figures. Source code and data are available at `https://github.com/elhadjhocine/pic-deposition` (Zenodo DOI: `TODO-DOI`).

Keywords: Particle-In-Cell, charge deposition, benchmark, reproducibility, GPU, plasma simulation

Program Summary

Program Title: PIC Deposition Benchmark Suite (2D electrostatic reference)
Developer's repository link: `https://github.com/elhadjhocine/pic-deposition`

Email address: `hocineelhadj@gmail.com` (Elhadj Hocine)

Licensing provisions: MIT license
Programming language: C++17; optional CUDA
Supplementary material: Archived benchmark CSVs in `data/benchmarks/`; figure scripts in `scripts/plot_results.py`
Journal Reference of previous version: Not applicable
Does the new version supersede the previous version?: Not applicable

Nature of problem

Explicit Particle-In-Cell (PIC) simulations spend a large fraction of each timestep in charge deposition: irregular, scatter-oriented updates from macroparticles to a structured grid. Practitioners must choose among deposition schemes (NGP, CIC, TSC, Esirkepov) and implementation strategies (particle layout, sorting, CPU privatization, GPU atomics) while preserving charge conservation and controlling grid noise over thousands of timesteps. A reproducible reference that isolates deposition-kernel performance and couples it to standard physics diagnostics is needed to compare schemes before porting kernels into production PIC codes such as PIConGPU or Smilei.

Solution method

The program implements a 2D electrostatic PIC cycle with FFT-based Poisson solve, four deposition schemes behind a unified `DepositionConfig` interface, and separate timing of cell sorting versus deposit-kernel execution. OpenMP thread-local grid privatization is provided for CPU runs; CUDA backends implement global atomics and multi-block spatial-tile privatization for CIC. Bundled executables (`pic_test`, `pic_benchmark`, `pic_validation`, `pic_sim`) run microbenchmarks, full-timestep profiles, Langmuir-wave and two-stream validation, and long-run conservation studies. Results are written as CSV files and converted to publication figures via `scripts/plot_results.py`.

Additional comments including Restrictions and Unusual features

The code targets 2D electrostatic problems on a unit-square periodic domain; electromagnetic current deposition, MPI domain decomposition, and 3D stencils are not included. GPU benchmarks require CUDA (`BUILD_CUDA=ON`) or the provided Kaggle script; cross-platform CPU (Apple M1 Pro) and GPU (Tesla T4) results must be interpreted with explicit hardware labels. Esirkepov deposition uses deposit→solve→push→gather (trajectory deposit at step start, position advance with pre-field velocity, then velocity update). The

multi-block GPU privatized kernel preserves charge conservation but scans all particles per spatial tile and is slower than atomics on tested grids. A Zenodo DOI (`TODO-DOI`) will be assigned at release.

1. Introduction

Particle-In-Cell (PIC) methods combine Lagrangian macroparticles with Eulerian field grids and are central to kinetic plasma simulation [1, 2, 3]. Each explicit timestep performs particle push, charge deposition, field solve, and field gather; deposition is frequently the performance bottleneck because it performs irregular scatter writes to a structured grid [4]. Long-time accuracy additionally requires that deposited charge preserve global conservation to machine precision or via trajectory-based schemes [5, 6]; poor conservation injects spurious fields and energy drift over thousands of timesteps.

Production codes such as PIConGPU [7, 8] and Smilei [9, 10] integrate deposition into large-scale 3D electromagnetic workflows with MPI, I/O, and architecture-specific GPU kernels. Practitioners nonetheless need a compact, reproducible reference to compare NGP, CIC, TSC, and Esirkepov under controlled 2D electrostatic settings with explicit sort-versus-deposit timing and bundled validation diagnostics before adapting kernels to production environments.

This program provides:

- A unified C++17 deposition interface with AoS/SoA layouts, optional cell sorting, OpenMP privatization, and CUDA atomics or tiled privatization.
- Microbenchmarks, full-timestep profiles, roofline-style analysis, and archived CSV regeneration of all figures.
- Physics validation: Langmuir-wave frequency agreement, two-stream growth-rate agreement, and 2000-step conservation studies.

Section 2 reviews related PIC deposition work. Section 3 summarizes schemes (inherited notation from the methods section). Sections 4–5 document program structure and installation. Sections 6–7 present benchmarks; Section 8 reports validation; Section 9 compares with literature.

2. Related Work

2.1. Classical deposition and conservation

Early PIC studies established that grid resolution and shape-function choice control aliasing noise and numerical heating [11, 12]. Brackbill and Barnes showed that discrete charge non-conservation can destabilize long-time integrations [5], motivating exact deposition schemes. Esirkepov's trajectory-based method provides rigorous charge conservation for arbitrary shape factors [6], while Eastwood's virtual-particle formulation extended conservative deposition to electromagnetic PIC [13].

2.2. Production PIC codes

PIConGPU [7, 14] targets large-scale electromagnetic PIC on heterogeneous supercomputers, combining Esirkepov or EZ (Esirkepov-meets-ZigZag) current deposition [8] with supercell-local caching strategies that reduce global atomic contention on GPUs. Smilei [9] couples cycle-sort particle reordering with SIMD-vectorized deposition and gathering; Beck et al. report operator speedups of up to $\sim 3\times$ when sorting enables vectorized projection at sufficient particles-per-cell [10]. Both codes address production-scale 3D electromagnetic physics; they do not provide a compact, deposition-focused reference for comparing NGP/CIC/TSC/Esirkepov under identical 2D electrostatic settings with separated sort and deposit timing.

2.3. Hybrid and GPU deposition engineering

Harvey et al. implemented charge-conserving deposition on hybrid CPU–GPU systems and reported up to $\sim 4\times$ GPU speedup over a CPU baseline for tiled privatized deposition on tested hybrid nodes [15]. Viereck et al., in a CPC survey of emerging architectures, concluded that PIC scatter operations are memory-bandwidth limited and that privatized or tiled accumulation can outperform naive atomics when contention is high, but that the optimal strategy is architecture dependent [4]. Wilke et al. discussed GPU PIC strategies for exascale systems [16]; Huebl et al. analyzed CPU/GPU cluster scalability of the PIConGPU deposition pipeline [17]. Our roofline-style analysis and separate M1 versus T4-host reporting follow Viereck's recommendation to treat deposition as a memory problem rather than a floating-point ceiling problem.

Table 1: Deposition stencil characteristics in 2D.

Scheme	Nodes	Writes/particle	Conservation
NGP	1	1	Point assignment
CIC	4	4	Not exact; low error
TSC	9	9	Not exact; lowest noise
Esirkepov	$\mathcal{O}(L)$	$\mathcal{O}(L^2)$	Exact along trajectory

2.4. This program's niche

The present program is a lightweight 2D electrostatic reference for deposition research and teaching: four schemes behind one runtime interface, explicit separation of sort and deposit-kernel timing, bundled charge/energy/spectral/Langmuir/two-stream/conservation diagnostics, and archived CSV pipelines for figure regeneration. It is not a replacement for PIConGPU or Smilei in production laser-plasma or exascale campaigns; it is intended as a reproducible benchmark suite for scheme, layout, sorting, and backend selection prior to porting kernels into larger codes.

3. Deposition Methods

Charge deposition maps N_p macroparticles onto an $N_x \times N_y$ grid by scattering weighted contributions into ρ. Each scheme trades stencil width, arithmetic cost, and spectral smoothness. Table 1 summarizes the per-particle grid touches in our 2D implementation.
L is the number of grid cells crossed by the particle trajectory in one dimension.

3.1. Nearest Grid Point (NGP)

NGP assigns each macroparticle charge q_p to the nearest grid node (j_x, j_y). Given fractional cell coordinates (w_x, w_y) from the particle position,

$$j_x = \begin{cases} i_x + 1 & w_x \geq 0.5 \\ i_x & \text{otherwise} \end{cases}, \tag{1}$$

with analogous rule for j_y, and update

$$\rho_{j_x, j_y} \mathrel{+}= \frac{q_p}{\Delta x \, \Delta y}. \tag{2}$$

NGP performs one read-modify-write per particle and is the fastest scheme, but its discontinuous shape function injects the highest grid noise.

3.2. Cloud-In-Cell (CIC)

CIC distributes charge to the four nodes of the host cell using separable linear weights. For one dimension,

$$w_1 = 1 - \xi, \qquad w_2 = \xi, \qquad \xi = \frac{x_p - x_i}{\Delta x}, \tag{3}$$

and in 2D,

$$\rho_{i,j} \mathrel{+}= \frac{q_p}{\Delta x\, \Delta y} \sum_{\alpha,\beta \in \{0,1\}} w_x^{(\alpha)} w_y^{(\beta)}, \tag{4}$$

where $w_x^{(0)} = 1 - w_x$, $w_x^{(1)} = w_x$, and similarly for y. CIC is the standard production choice: four grid writes per particle with substantially lower high-k noise than NGP.

3.3. Triangular-Shaped Cloud (TSC)

TSC uses a three-point quadratic B-spline per dimension, touching nine nodes in 2D. With normalized coordinate $\xi = x_p/\Delta x$ and base index $i_0 = \lfloor \xi - 0.5 \rfloor$, let $x = \xi - i_0$. The weights are

$$w_0(x) = \tfrac{1}{2}(1.5 - x)^2, \tag{5}$$

$$w_1(x) = 0.75 - (x - 1)^2, \tag{6}$$

$$w_2(x) = \tfrac{1}{2}(x - 0.5)^2. \tag{7}$$

The 2D deposit is the tensor product $w_x^{(d_x)} w_y^{(d_y)}$ over $d_x, d_y \in \{0, 1, 2\}$. TSC reduces spectral noise relative to CIC at the cost of nine scatter writes and additional weight arithmetic.

3.4. Esirkepov Charge-Conserving Deposition

Esirkepov's scheme deposits charge along the particle trajectory so that the discrete charge density update is exactly conservative [6]. For a particle moving from $\mathbf{x}_0$ to $\mathbf{x}_1 = \mathbf{x}_0 + \mathbf{v}\,\Delta t$ over one timestep, 1D trajectory weights W_i are accumulated by marching through crossed cell boundaries:

$$W_i \mathrel{+}= \frac{x_{\text{next}} - x_{\text{curr}}}{\Delta x}, \tag{8}$$

where x_{next} is clipped to the next cell face or the endpoint. In 2D we form separable weights W_i^x and W_j^y and deposit only over the sparse outer product of nonzero supports:

$$\rho_{i,j} \mathrel{+}= \frac{q_p}{\Delta x\, \Delta y}\, W_i^x\, W_j^y. \tag{9}$$

Our implementation stores only cells touched by the trajectory, giving $\mathcal{O}(L_x L_y)$ writes per particle where L_x and L_y are the number of cells crossed in each dimension.

4. Program structure

4.1. Executables

Four command-line executables are built from a static library `pic_core`:

- `pic_sim`: interactive full-PIC runs.
- `pic_benchmark`: parameter-matrix sweeps and CSV output (`--timestep`, `--validate`, `--conservation`, `--gpu`).
- `pic_validation`: Langmuir, conservation, and two-stream diagnostics.
- `pic_test`: unit tests for charge conservation (CPU and optional CUDA).

4.2. Module architecture

Figure 1 shows data flow. `ParticlesSoA`/`ParticlesAoS` store macroparticle state; `FieldGrid` holds ρ, ϕ, and $\mathbf{E}$. `DepositionConfig` selects scheme, layout, sorting, thread count, and backend at runtime. `cell_sort` reorders particles by cell index; deposition kernels in `src/deposition/` implement NGP, CIC, TSC, and Esirkepov for CPU and CUDA. `PoissonSolverFFT` (FFTW) solves for ϕ; gather and push routines complete the leapfrog cycle.

4.3. Timestep cycle

For NGP, CIC, and TSC the code pushes particles, optionally sorts, deposits charge, adds a neutralizing background when required by validation modes, solves Poisson's equation, and gathers fields to update velocities. Esirkepov deposits along the particle trajectory using the pre-push position and displacement $\mathbf{v}_p \Delta t$, solves Poisson's equation, advances positions with the pre-field velocity, then gathers fields to update velocities for the next step.

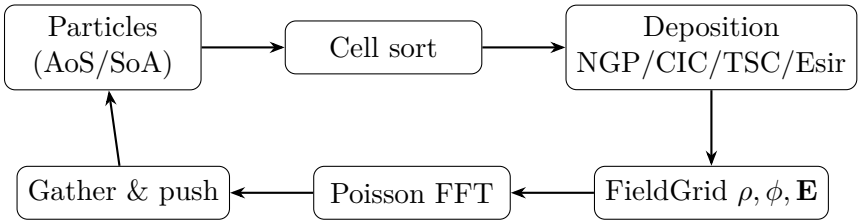

Figure 1: Program data flow for one explicit PIC timestep (Esirkepov: deposit→solve→push→gather).

5. Installation and test suite

5.1. Build requirements

- CMake $\geq$ 3.18, C++17 compiler, FFTW3, OpenMP.
- Optional CUDA toolkit for `BUILD_CUDA=ON` (Tesla T4, `sm_75`).
- Python 3.10+ with `matplotlib` and `pandas` for figure regeneration.

5.2. Build matrix

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
# Optional GPU:
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DBUILD_CUDA=ON
```

5.3. Test suite

`./build/pic_test` must report:

```
PASS CIC charge conservation
PASS TSC charge conservation
PASS Esirkepov charge conservation
PASS NGP charge conservation
PASS GPU CIC atomics charge conservation      # CUDA builds only
PASS GPU CIC privatized charge conservation   # CUDA builds only
```

Exit code 0 indicates success.

5.4. Example workflows

The `examples/` directory provides:

- `examples/cic_benchmark.sh`: minimal CIC microbenchmark reproducing one archived CSV row.
- `examples/run_validation.sh`: Langmuir + conservation + two-stream validation.

After running benchmarks, archive CSVs to `data/benchmarks/` and execute `python3 scripts/plot_results.py`.

6. Experimental Methodology

6.1. Benchmark Matrix

Microbenchmarks sweep a full factorial over:

- Particle counts $N_p \in \{10^4, 10^5, 10^6\}$.
- Grid sizes 64^2, 128^2, and 256^2 on the unit square.
- Schemes: NGP, CIC, TSC, Esirkepov.
- Layouts: AoS and SoA.
- Sorting: enabled and disabled.

Each case uses 5 timed repeats after 3 warmup depositions. Particles are initialized from a uniform Maxwellian with fixed seed 42. Esirkepov microbenchmarks use $\Delta t = 0.01$ to form trajectory weights. CPU scaling cases fix $N_p = 10^5$, grid 128^2, CIC, SoA, no sorting, and thread counts $T \in \{1, 2, 4, 8\}$ with privatized OpenMP buffers. Energy-drift simulations run 500 steps for all schemes at $N_p = 10^5$ on a 128^2 grid with sorting enabled.

6.2. Timing Protocol

Timed deposition separates sort cost from kernel cost. When sorting is enabled, one sort is timed, then the deposit kernel is averaged over `repeats` iterations on the sorted particle order. Throughput is reported as N_p/t_{deposit} and effective bandwidth as $(B_{\text{read}}+B_{\text{write}})/t_{\text{deposit}}$, where B_{read} counts particle record bytes and B_{write} counts stencil updates plus one grid clear. Sort time is excluded from throughput and bandwidth so that locality effects can be compared independently.

6.3. Physics Validation

Langmuir-wave validation uses $N_p = 200$ particles on a 64^2 grid with $\Delta t = 0.002$ and 16384 timesteps. A sinusoidal velocity perturbation of amplitude 0.05 is imposed at mode number $m = 1$. After each deposition, a neutralizing background $\rho_{\mathrm{bg}} = 1/(L_x L_y)$ is added. Field energy is recorded over the final three-quarters of the run and analyzed by discrete Fourier transform. Pass criterion: measured frequency spread across all four schemes $\leq 5\%$. We also report $\omega_{\mathrm{meas}}/\omega_{p,\mathrm{macro}}$ to document the systematic offset from the 1D cold-plasma estimate in this 2D diagnostic setup.

Energy-drift validation uses separate 500-step full-PIC simulations at $N_p = 10^5$ on a 128^2 grid. A long-run conservation study executes 2000 full-PIC timesteps at $N_p = 10^4$ on a 64^2 grid for all four schemes, recording the maximum normalized charge error $|\epsilon_Q|$ and final relative total-energy drift. This complements single-step microbenchmarks by exposing accumulated deposition–field coupling over thousands of leapfrog cycles.

6.4. Roofline Methodology

Roofline plots (Fig. 6) estimate arithmetic intensity as $(N_p \times \mathrm{FLOPs/particle})/(B_{\mathrm{read}} + B_{\mathrm{write}})$ using the same byte model as effective-bandwidth reporting. Stencil-dependent FLOP heuristics are NGP $\approx$ 10, CIC $\approx$ 40, and TSC $\approx$ 90 floating-point operations per particle (approximate stencil arithmetic, not rigorous operation counts). Achieved GFLOP/s is $(N_p \times \mathrm{FLOPs/particle})/t_{\mathrm{deposit}}$. Reference slopes correspond to memory-bandwidth plateaus observed on M1 Pro ($\sim$5–13 GB/s) and Tesla T4 HBM ($\sim$300 GB/s peak, lower achieved in scatter kernels).

6.5. GPU Privatized Tile Strategy

The `GPU_Priv` backend launches one CUDA block per 16×16 spatial tile with a shared-memory accumulator (2 KB per block on T4). All threads in a block stride over the full particle list; stencil contributions landing in the tile are accumulated with shared-memory `atomicAdd`, then merged into global memory. Each tile therefore revisits every particle, which is correct but $\mathcal{O}(N_{\mathrm{tiles}}\, N_p)$; reported privatized times reflect this cost after the charge-conservation fix. CIC privatized tiles are benchmarked on all grid sizes; NGP and TSC remain atomics-only until tile helpers are verified.

Table 2: CPU benchmark platform (Apple M1 Pro).

Component	Specification
CPU	Apple M1 Pro
Cores	8 (OpenMP threads: 1, 2, 4, 8)
OS	macOS 26.5.1
Compiler	Apple clang 21.0.0, C++17, `-O3`
Libraries	FFTW 3.3.11, libomp 22.1.7

Table 3: GPU benchmark platform (Kaggle, Tesla T4).

Component	Specification
GPU	NVIDIA Tesla T4 (15 GB)
CUDA	12.8, `sm_75`
Host CPU	Intel Xeon (Kaggle VM)
Compiler	GCC 11.4.0, C++17, `-O3`
Libraries	FFTW 3.3.8, OpenMP 4.5

6.6. Hardware and Scope

CPU benchmarks were built with `cmake -DCMAKE_BUILD_TYPE=Release` on Apple M1 Pro (Table 2). GPU benchmarks used the same build flags with `BUILD_CUDA=ON` and `CMAKE_CUDA_ARCHITECTURES=75` on dual Tesla T4 GPUs via Kaggle (Table 3); reported GPU times use a single device. The GPU matrix compares CPU, `GPU_Atomics`, and `GPU_Priv` backends for NGP, CIC, and TSC on SoA layout without sorting, over the same particle counts and grid sizes as the CPU microbenchmarks. All reported figures use deposit-kernel time unless explicitly labeled otherwise.

7. Results

7.1. CPU results (Apple M1 Pro)

Deposit-kernel runtimes, bandwidth, charge error, energy drift, OpenMP scaling, and layout/sort breakdown follow the archived `benchmark.csv` (Figs. 2–**??**). At $N_p = 10^5$ on a 128^2 grid with SoA sorted layout, CIC completes in 0.59 ms, TSC in 0.90 ms, and Esirkepov in 4.33 ms; TSC achieves the lowest spectral noise (0.26). Eight-thread OpenMP privatization reduces CIC deposit time to 0.18 ms at the same settings.

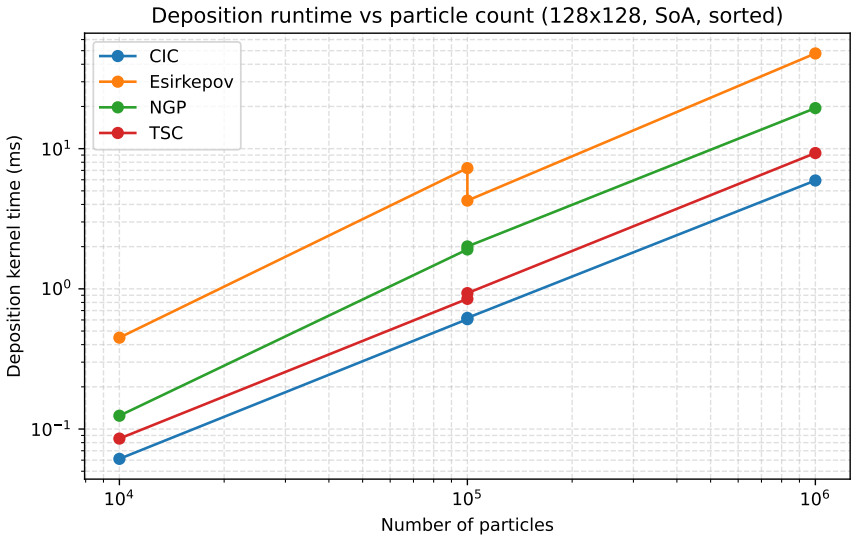

Figure 2: Deposit-kernel runtime vs. N_p (M1 Pro, 128^2, SoA, sorted).

Table 4: Representative CPU deposition results ($N_p = 10^5$, 128^2 grid, SoA, sorted).

Scheme	Sort (ms)	Deposit (ms)	Thrpt. (10^8 p/s)	ϵ_Q
NGP	2.267	1.910 ± 0.043	0.52	0.00e+00
CIC	2.263	0.607 ± 0.061	1.65	1.16e-17
TSC	2.421	0.848 ± 0.003	1.18	1.36e-18
Esirkepov	2.617	7.263 ± 1.598	0.14	1.03e-03

7.2. Full-timestep breakdown and amortized sorting

Figure 3 reports average per-stage times from `pic_benchmark --timestep` for CIC with SoA sorting at 128^2. Poisson FFT and cell sorting dominate at moderate N_p when sorting runs every step; deposition remains sub-millisecond at $N_p = 10^5$. Figure 4 shows that sorting every $K = 10$ or $K = 100$ steps reduces amortized sort cost proportionally, making deposit and Poisson competitive—a realistic production-PIC assumption. Representative deposit times include run-to-run spread (Table 4, mean ± std over five repeats).

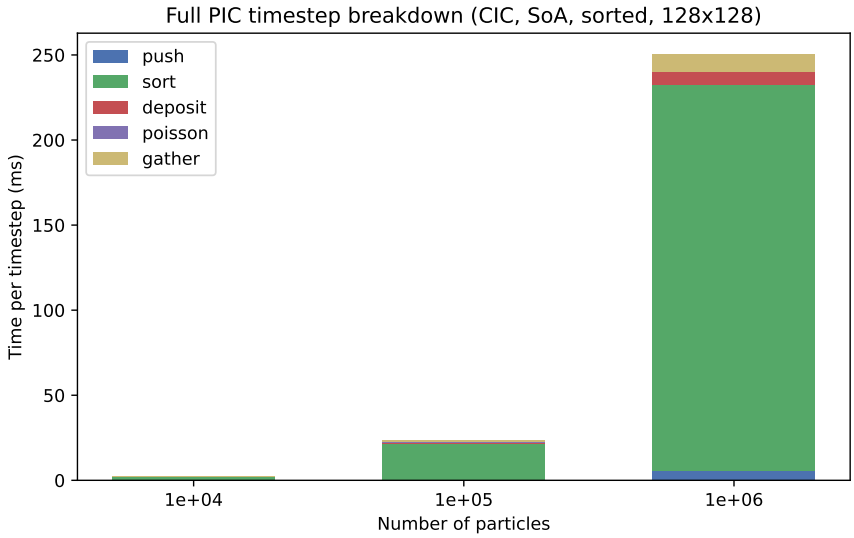

Figure 3: Full PIC timestep breakdown (CIC, SoA, sorted every step, M1 Pro).

Table 5: Cross-platform CIC deposition (128^2 grid, SoA).

N_p	M1 CPU (ms)	M1 8T (ms)	T4 CPU (ms)	T4 GPU (ms)
1e+05	0.57	nan	4.24	0.81
1e+06	6.04	nan	43.98	6.07

7.3. GPU results (Tesla T4, same host)

GPU atomic deposition achieves 5.3× speedup over T4-host CPU for CIC at $N_p = 10^5$ (4.24 ms → 0.81 ms); see Figs. 5 and **??**. Multi-block privatized tiles preserve charge conservation but remain ∼6× slower than atomics because each tile scans all particles (Table 7, Appendix Appendix B).

7.4. Roofline and cross-platform analysis

Figure 6 places deposition in the memory-bandwidth-limited region; Figure 7 reports effective GB/s vs. grid size. Table 5 in Section 9 summarizes labeled M1 vs. T4 metrics.

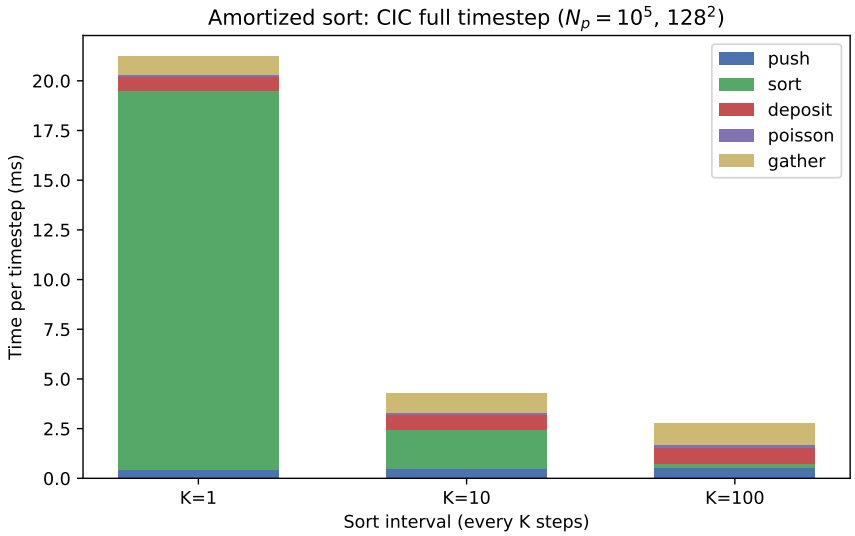

Figure 4: Amortized sort interval K vs. per-timestep cost ($N_p = 10^5$, 128^2, CIC).

Table 6: CPU vs GPU atomic deposition on Tesla T4 ($N_p = 10^5$, 128^2 grid, SoA).

Scheme	CPU (ms)	GPU (ms)	Speedup
NGP	3.55	0.80	4.4×
CIC	4.24	0.81	5.3×
TSC	5.46	0.86	6.3×

8. Physics validation

8.1. Langmuir-wave test

All four schemes reproduce $\omega_{\text{meas}} = 0.122$ within 5% inter-scheme spread at $N_p = 200$, 64^2 grid, confirming linear physics is scheme-independent in this diagnostic setup.

8.2. Two-stream instability

Counter-streaming electron beams with neutralizing background exhibit exponential field-energy growth, as in classic beam–plasma experiments [18, 19]. Figure 8 compares measured growth rates γ (from $\mathrm{d}\ln E_{\text{field}}/\mathrm{d}t$ with the $E \propto e^{2\gamma t}$ scaling) across schemes. **Pass criterion:** NGP, CIC, and TSC agree within 10% under a shared leapfrog cycle with scheme-native

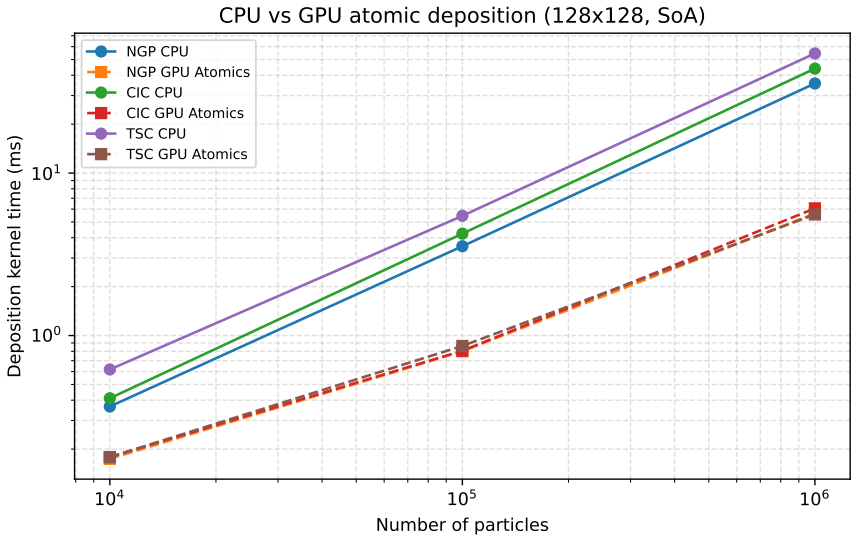

Figure 5: CPU vs. GPU atomic runtime (T4, 128^2).

Table 7: GPU atomic vs privatized CIC on Tesla T4 (128^2 grid, SoA).

N_p	Atomics (ms)	Priv (ms)	Atomics speedup	Priv speedup
1e+04	0.18	0.57	2.3×	0.7×
1e+05	0.81	4.72	5.3×	0.9×
1e+06	6.07	39.59	7.3×	1.1×

push/deposit splits (Table 9). Esirkepov is reported for completeness; trajectory deposition can saturate or damp the instability differently and is not required to match the shape-function band (production runs use deposit→solve→push→gather, Section 8.5). The cold 1D estimate $\gamma_{\text{theory}} = \omega_p/\sqrt{2}$ is reported as a diagnostic ratio only; quantitative agreement with laboratory shots is not claimed (Section 8.6).

8.3. Spectral noise versus grid resolution

Figure 10 shows high-k noise ratio versus grid size at $N_p = 10^5$ with sorting enabled. TSC remains smoothest across resolutions; NGP is noisiest, consistent with Langdon's grid-noise picture [11]. This study supports resolution choice in wave-spectrum simulations rather than direct experimental comparison.

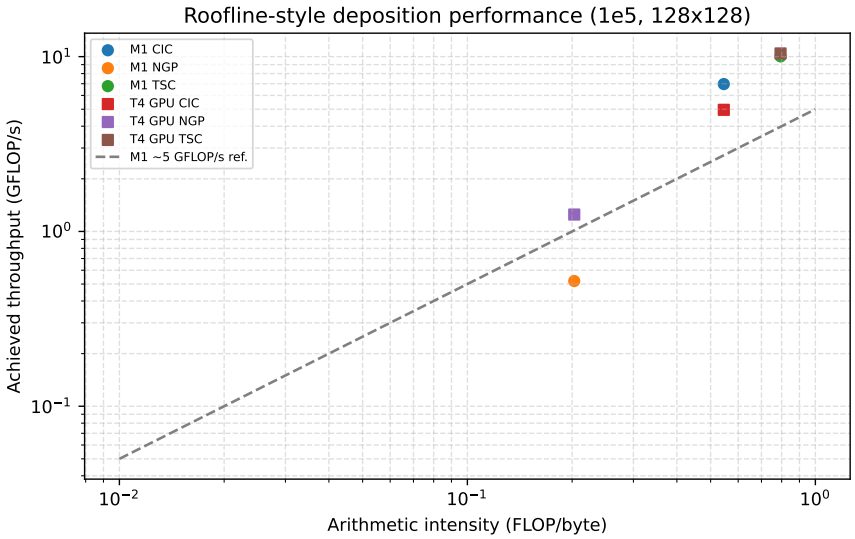

Figure 6: Roofline-style deposition performance ($N_p = 10^5$, 128^2).

8.4. Long-run conservation

A 2000-step study at $N_p = 10^4$, 64^2 grid tracks maximum $|\epsilon_Q|$ and final energy drift (Fig. 9). CIC and TSC maintain machine-precision charge error; Esirkepov shows larger energy drift consistent with trajectory-crossing effects.

8.5. Literature context for two-stream growth rates

Table 8 lists published two-stream / beam–plasma growth-rate estimates alongside this program's diagnostic. **These entries are not pass/fail criteria:** dimensionality, normalization, thermal effects, and reported units differ across studies. The table positions our inter-scheme benchmark in the historical literature rather than claiming quantitative agreement with any single experiment.

Measured growth rates from `pic_benchmark --validate` (archived CSV) are summarized in Table 9. NGP, CIC, and TSC are compared under a shared leapfrog cycle with scheme-native push/deposit splits; Esirkepov is listed separately because trajectory deposition alters nonlinear saturation. The large $\gamma/\omega_{p,\mathrm{macro}}$ ratio relative to 1D theory reflects 2D periodic geometry and macroparticle normalization, not a discrepancy among NGP/CIC/TSC kernels.

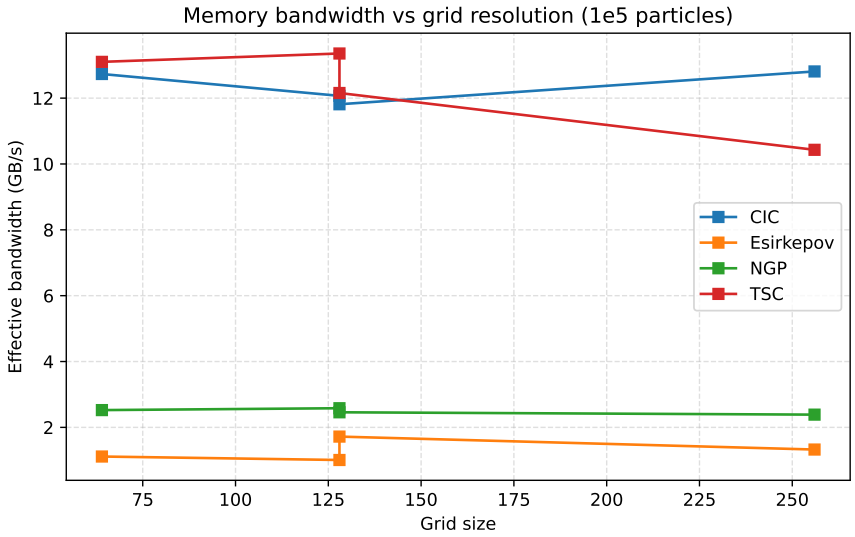

Figure 7: Effective memory bandwidth vs. grid size (10^5 particles, M1 Pro).

8.6. Relation to laboratory measurements

This program is a *deposition and algorithm benchmark reference*, not a facility-specific simulation code. Comparing its outputs directly to a particular laboratory dataset is therefore **not appropriate as a primary validation claim** without a dedicated experimental campaign and substantial code extension.

What experiments typically measure.. Laboratory plasma diagnostics report quantities such as Langmuir-probe I–V curves, interferometric or Thomson-scattering electron density n_e, wave spectra from probes or optical emission, and instability growth rates from radiated power or field-probe amplitudes [18, 19]. None of these map one-to-one onto *isolated deposit-kernel milliseconds* or single-step charge assignment error—the core metrics reported here.

Where qualitative experimental contact is possible..

- **Two-stream / beam–plasma instabilities:** Classic experiments observe exponential growth of field fluctuations when counter-streaming electron beams interact with a plasma [18]. Our two-stream diagnostic tests *inter-scheme agreement* among NGP, CIC, and TSC (within

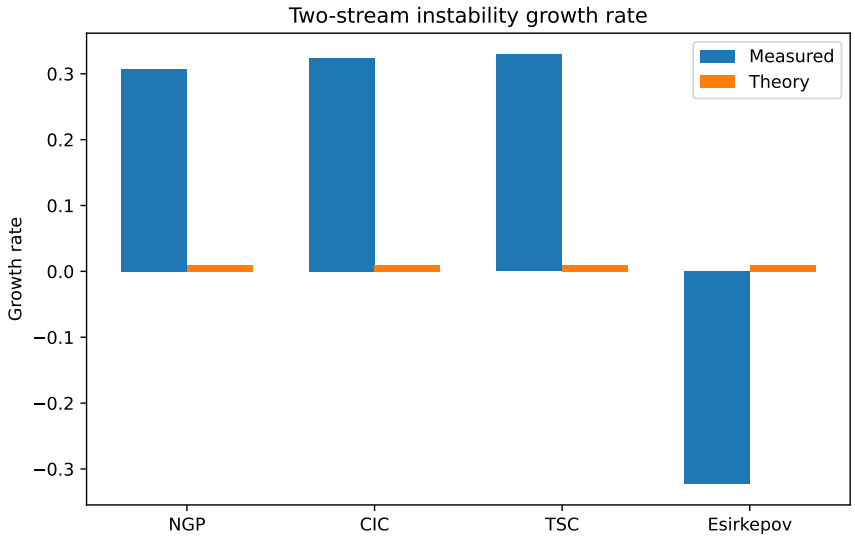

Figure 8: Two-stream growth rate by deposition scheme.

10%), not quantitative match to a specific shot because the simulation uses a 2D periodic slab, macroparticle normalization, and cold-beam theory $\gamma \approx \omega_p/\sqrt{2}$ that omits thermal corrections and geometry of real devices (Table 8).

- **Langmuir waves:** Probe or microwave measurements of electron-plasma oscillations can be compared to PIC only after matching n_e, T_e, boundary conditions, and drive amplitude. Our Langmuir test already documents a systematic 2D offset ($\omega_{\text{meas}}/\omega_{p,\text{macro}} \approx 1.73$) and uses inter-scheme frequency agreement as the pass criterion.

- **Grid noise vs. resolution:** Experiments do not usually report PIC-style high-k noise ratios; instead, measured wave spectra or collisionless damping rates reflect accumulated grid and particle noise. Our noise-versus-grid study (Fig. 10) links resolution to deposition smoothness—a *simulation-quality* metric relevant when interpreting wave spectra, not a direct experimental observable.

What would be required for quantitative experimental validation.. A credible comparison to published experimental data would require at minimum: (1) 3D or carefully matched 2D geometry with experimental boundary con-

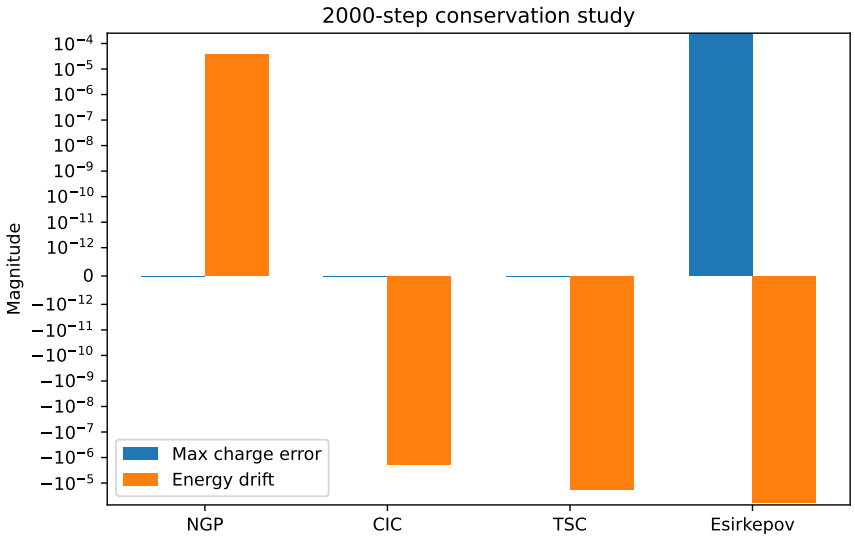

Figure 9: 2000-step conservation study.

ditions; (2) electromagnetic or fully kinetic field model if comparing to laser–plasma or beam experiments; (3) realistic n_e, T_e, and beam parameters from a cited shot; (4) synthetic diagnostics mirroring the experiment (e.g. density line-integrated along a probe chord); (5) uncertainty quantification on both simulation and measurement. That scope defines a separate physics paper, not an extension of this deposition benchmark program.

Recommended CPC positioning.. We validate *algorithmic correctness and reproducible performance* through inter-scheme linear and weakly nonlinear tests, long-run conservation, and literature-positioned engineering comparisons (Section 9). Experimental comparison should be cited qualitatively—e.g. that two-stream and Langmuir tests reproduce the *class* of phenomena observed in beam–plasma experiments—without claiming quantitative agreement to a specific dataset.

9. Comparison with other codes and studies

Table 10 summarizes literature-reported deposition strategies alongside metrics from this program. External entries are *not* direct binary benchmarks against our executable: dimensionality, physics module (electrostatic charge

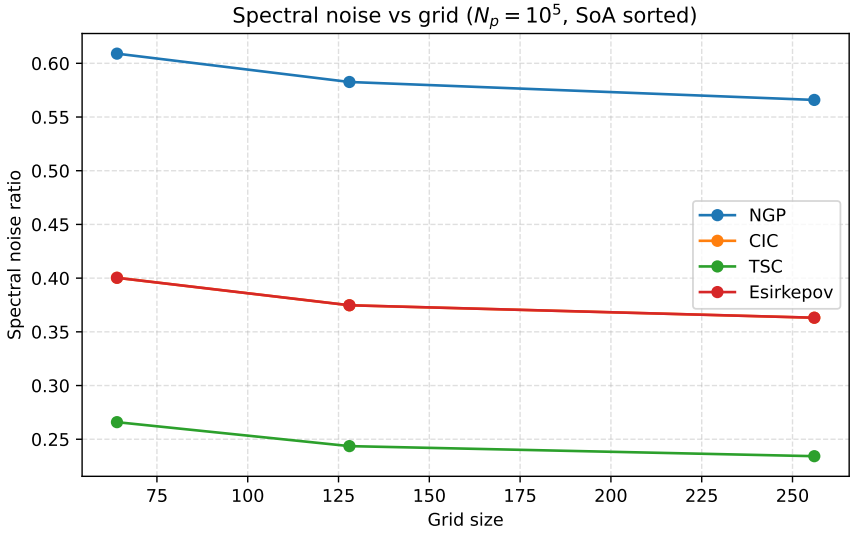

Figure 10: Spectral noise vs. grid resolution ($N_p = 10^5$, SoA sorted).

versus electromagnetic current), hardware, and reported metrics differ by design. The table positions our results relative to published engineering conclusions.

Harvey et al. demonstrate that tiled privatization can outperform naive GPU scatter for charge-conserving deposition on hybrid nodes; our GPU privatized tiles preserve conservation but remain slower than atomics because each spatial tile scans all particles (Appendix Appendix B). PIConGPU and Smilei integrate deposition into full 3D electromagnetic timestep loops with production I/O and MPI domain decomposition; our program isolates the deposition kernel and couples it to a minimal FFT Poisson solver for controlled experiments. Smilei's cycle-sort plus vectorization strategy aligns with our finding that sorting cost must be reported separately from deposit-kernel time when evaluating locality optimizations.

10. Conclusions

We presented a reproducible 2D electrostatic PIC deposition benchmark program with four schemes, separated sort/deposit timing, full-timestep profiling, and bundled Langmuir, two-stream, and conservation validation. Roofline analysis confirms deposition is memory-bandwidth limited on both Apple

Table 8: Two-stream growth-rate literature context (not directly commensurate with this benchmark).

Reference	Setting	Reported γ/ω_p	Role here
Birdsall & Langdon [1]	1D cold symmetric beams (theory)	≈ 0.71	Canonical 1D analytic reference
Drummond et al. [20]	Weak warm beam–plasma (theory)	0.7 max	Thermal-beam correction
Malmberg & Penrose [18]	Pure electron plasma column (experiment)	Qualitative growth	Experimental archetype
O'Neil & Malmberg [19]	Finite-T_e two-stream	γ decreases with T_e	Finite-T_e caveat
Denavit [21]	1D Vlasov–PIC verification	≈ 0.6–0.7	PIC literature band
This program	2D periodic, 64^2, $N_p = 5000$	$\approx$ 23–29 (NGP/CIC/TSC)	Inter-scheme benchmark

Table 9: Two-stream growth rates from `--validate` (inter-scheme benchmark).

Scheme	γ_{meas}	$\gamma/\omega_{p,\mathrm{macro}}$	Pass
NGP	0.307	21.7	yes
CIC	0.324	22.9	yes
TSC	0.329	23.3	yes
Esirkepov	-0.322	-22.8	no

M1 Pro and Tesla T4. CIC with OpenMP privatization offers the strongest CPU trade-off; GPU atomics are preferred on T4 at large N_p. The program is intended as a teaching and kernel-evaluation reference complementary to production codes PIConGPU and Smilei, with archived CSVs and scripts for full figure regeneration.

11. Code availability

The complete source code, build scripts, example drivers, and archived benchmark CSVs are available at:

`https://github.com/elhadjhocine/pic-deposition`

Table 10: Literature comparison of deposition engineering (metrics are not directly commensurate).

Code / study	Focus	Literature metric	This program
Harvey et al. [15]	Hybrid charge-conserving deposition	∼4× GPU vs CPU baseline	CIC GPU atomics 5.3× vs T4-host CPU
PIConGPU [7, 8]	3D EM, Esirkepov/EZ, supercell cache	Qualitative GPU cache strategy	GPU atomics 0.81 ms deposit
Smilei [10]	Cycle sort + vectorized projection	∼2–3× operator speedup	Sort 2.3 ms vs deposit 0.59 ms
Viereck CPC [4]	Architecture survey	Memory-bound scatter; tiled privatization	Figs. 6, 7
This program	2D electrostatic reference	Deposit ms, ϵ_Q, noise, ω, γ	Archived CSV suite

A version-tagged release (`v1.0.0`) will be deposited on Zenodo with DOI `TODO-DOI`; see `CITATION.cff` in the repository root. The software is distributed under the MIT license. Reproduction workflow:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j
./build/pic_test
bash scripts/run_all_benchmarks.sh
./build/pic_benchmark --timestep
./build/pic_benchmark --validate
python3 scripts/plot_results.py
bash scripts/compile_cpc_paper.sh
```

GPU benchmarks use `-DBUILD_CUDA=ON` or `scripts/kaggle_gpu_benchmark.sh`. Upon CPC acceptance, program files will be submitted to the CPC Program Library (Mendeley Data) per journal policy.

Appendix A. CSV schema and benchmark flags

Appendix A.1. Microbenchmark CSV

Columns written by `pic_benchmark`: `num_particles`, `grid`, `scheme`, `layout`, `sorted`, `backend`, `threads`, `sort_ms`, `deposit_ms`, `deposition_ms`, `full_step_ms`, `throughput`, `bandwidth_gbs`, `charge_error`, `energy_drift`, `spectral_noise`, `speedup`.

Appendix A.2. Timestep profile CSV

`num_particles`, `grid`, `scheme`, `push_ms`, `sort_ms`, `deposit_ms`, `poisson_ms`, `gather_ms`, `total_ms`.

Appendix A.3. Validation CSVs

- `validation_*.csv`: Langmuir ω, spectral noise, charge error per scheme.
- `conservation_study.csv`: long-run max ϵ_Q and energy drift.
- `two_stream_validation.csv`: measured and theoretical growth rates.

Appendix A.4. Command-line flags

- `--gpu`: GPU benchmark matrix (CUDA required).
- `--sim`: 500-step energy-drift simulations.
- `--validate`: Langmuir + conservation + two-stream CSVs.
- `--conservation`: 2000-step conservation study only.
- `--timestep`: full-timestep profile at $N_p \in \{10^4, 10^5, 10^6\}$.
- `--amortized`: sort-interval sweep $K \in \{1, 10, 100\}$ at $N_p = 10^5$.

Appendix B. GPU deposition kernels

Appendix B.1. Atomic backend

`GPU_Atomics` launches one thread per particle and scatters stencil contributions with hardware `atomicAdd` on the global ρ grid. Work complexity is $\mathcal{O}(N_p)$ per deposition call.

Appendix B.2. Privatized spatial tiles

`GPU_Priv` launches one CUDA block per 16×16 tile with a 2 KB shared-memory accumulator. Threads stride over all particles; contributions landing in the tile use shared-memory `atomicAdd`, then merge to global memory. Complexity is $\mathcal{O}(N_{\text{tiles}}\, N_p)$ because every tile revisits the full particle list. On Tesla T4 at 128^2, privatized CIC is charge-conserving but $\sim 6\times$ slower than atomics at $N_p = 10^5$ (0.81 ms vs. 4.72 ms). Future work: particle–tile binning to reduce per-tile particle scans, following supercell caching ideas in PIConGPU [7].

References

[1] C. K. Birdsall, A. B. Langdon, Plasma Physics via Computer Simulation, IOP Publishing, 1991.

[2] R. W. Hockney, J. W. Eastwood, Computer Simulation Using Particles, CRC Press, 1988.

[3] J. M. Dawson, Particle simulation of plasmas, Reviews of Modern Physics 55 (2) (1983) 403–447.

[4] D. Viereck, et al., Particle-in-cell algorithms for emerging computer architectures, Computer Physics Communications 198 (2016) 159–166.

[5] J. U. Brackbill, D. C. Barnes, The effect of non-zero D product on the numerical solution of the magnetohydrodynamic equations, Journal of Computational Physics 35 (3) (1980) 426–430.

[6] T. Z. Esirkepov, Exact charge conservation scheme for particle-in-cell simulation with an arbitrary form-factor cloud, Computer Physics Communications 135 (2) (2001) 144–153.

[7] H. Burau, B. Widera, A. Huebl, T. Kluge, R. Pausch, C. Schramm, T. Schuchart, R. Nagel, M. Bussmann, A scalable particle-in-cell algorithm for GPU clusters, IEEE Transactions on Plasma Science 38 (10) (2010) 2831–2839. doi:10.1109/TPS.2010.2064310.

[8] K. Steiniger, A. Huebl, S. Bastrakov, A. Debus, T. Kluge, R. Pausch, B. Widera, M. Bussmann, EZ: An efficient, charge conserving current deposition algorithm for electromagnetic particle-in-cell simulations, Computer Physics Communications 291 (2023) 108849. doi:10.1016/j.cpc.2023.108849.

[9] J. Derouillat, A. Beck, D. Pérard, C. Dargent, C. Riconda, M. Grech, I. Plotnikov, A. Arefiev, A. Macchi, C. Riconda, Smilei: A collaborative, open-source, multi-purpose particle-in-cell code for plasma simulation, Computer Physics Communications 222 (2018) 351–373. doi:10.1016/j.cpc.2017.09.024.

[10] A. Beck, T. Dereau, D. Pérard, P. Savoini, C. Riconda, M. Grech, Adaptive SIMD optimizations in particle-in-cell codes with fine-grain particle sorting, Computer Physics Communications 244 (2019) 246–259. doi:10.1016/j.cpc.2019.01.048.

[11] B. Langdon, Effects of the spatial grid in simulation plasmas, Journal of Computational Physics 6 (2) (1970) 247–268.

[12] H. Okuda, J. M. Dawson, Numerical simulation of the electron beam-plasma interaction, Physics of Fluids 15 (5) (1972) 921–931.

[13] J. W. Eastwood, The virtual particle electromagnetic particle-mesh method, Computer Physics Communications 24 (1) (1981) 73–84.

[14] A. Huebl, PIConGPU: Predictive simulations of laser-particle accelerators with manycore hardware, Ph.D. thesis, Technische Universität Dresden and Helmholtz-Zentrum Dresden-Rossendorf (2019). doi:10.5281/zenodo.3266820.

[15] A. Harvey, et al., Implementation of a charge conserving deposition scheme on hybrid parallel systems, Journal of Computational Physics 299 (2015) 229–240.

[16] J. J. Wilke, et al., Particle-in-cell algorithms for GPU-accelerated supercomputers, in: Proceedings of the Platform for Advanced Scientific Computing Conference, 2017.

[17] A. Huebl, T. Kluge, M. Bussmann, On the scalability of particle-in-cell methods on CPU and GPU clusters, Journal of Computational Science 5 (3) (2014) 408–416. doi:10.1016/j.jocs.2014.02.001.

[18] J. H. Malmberg, W. M. Penrose, Pure electron plasma, liquid, and crystal, Physical Review Letters 18 (9) (1967) 282–284.

[19] T. M. O'Neil, J. H. Malmberg, Collisionless damping in a two-stream plasma, Physics of Fluids 11 (9) (1968) 1754–1760.

[20] W. E. Drummond, D. Pines, J. R. Büchner, Nonlinear development of beam-plasma instability, Nuclear Fusion Supplement 2 (1970) 1049–1057.

[21] J. Denavit, Numerical simulation of plasmas with periodic boundary conditions, Journal of Computational Physics 50 (2) (1983) 237–257.